

An Introduction to TensorFlow Programming in Python



Deep learning is now widely used for the development of intelligent systems and has become a powerful tool for Big Data analysis. TensorFlow is the leading open source software for deep learning and is used for computer based natural language processing (NLP), computer vision, speech recognition, fault diagnosis, predictive maintenance, mineral exploration and much more.

This article will acquaint readers with the basic environment of TensorFlow, its computational library, and with dataflow graphs, which will help them with its advanced applications. Since TensorFlow's computational environment is graph based processing, it is of utmost necessity to understand it at the outset. Here we shall learn how graph based computing in TensorFlow is performed within a Python Anaconda environment. TensorFlow building blocks are constants, placeholders and variables, all of which form the graph based machine learning computation environment where computing operations interact with each other.

A higher level TensorFlow API assists in building prototype models, but the knowledge of lower level TensorFlow core is valuable for experimentation and debugging code. It gives an inner view of the operation model of the code, which helps us to understand the code using higher level APIs.

Graphs and sessions

TensorFlow uses a dataflow graph to represent all computations in terms of the dependencies between individual operations. At the outset, programming requires a dataflow graph to define all operations, after which a TensorFlow session is created to run parts of the graph across a set of local and remote devices. High level APIs such as *tf.estimator.Estimator* and Keras hide the details of graphs and sessions from the end user. Low level programming is useful to understand how the graph model works under the process session.

In a dataflow graph, the nodes represent units of computation and the edges represent the data consumed

or produced by a computation. For example, in a matrix multiplication operation, *tf.matmul* is a node in a TensorFlow graph and the multiplicand, multiplier and the result of multiplication are the three edges. This dataflow graph processing environment is helpful for distributed and parallel computing. It also speeds up the compilation process to convert the graph to code. Since the dataflow graph is a language-independent representation of the entire process environment, it is portable amongst different programming platforms. The dataflow representation of a process environment can be saved in one programming environment and transported to another programming environment.

The TensorFlow token *tf.Graph* contains dual information — the graph structure and graph collection. Using nodes and edges, the graph structure represents the composition of each individual operation, but it does not prescribe the methods of their use. Graph collections, on the other hand, provide a general mechanism for storing a collection of metadata within the graph. The graph collection mechanism provides ways to add and view relevant objects to and from key fields.

Creating a graph

In general, a TensorFlow program starts with a graph building phase. During this process, the API adds a new node and edge to a default graph instance. For example, a *tf.constant(x)* creates a single operation to produce *x*, then adds the value to the default graph and returns a tensor that represents the constant value. In the case of a variable, *tf.Variable(0)* adds an operation to store a writeable tensor value. The variable survives in between two session runs (*tf.Session.run*). If we